

Hệ thống phát hiện xâm nhập phân tán thời gian thực trên cụm thiết bị biên Jetson Orin Nano Super với Kafka và PySpark

Nguyễn Vũ Thái¹ Bùi Văn Dũng² Đỗ Trí Nhựt² Nguyễn Văn Dũ³

SOICT 2026

¹Phân hiệu tại TP. Hồ Chí Minh, Trường Đại học Giao thông Vận tải, TP. Hồ Chí Minh, Việt Nam

²Trường Đại học Công nghệ Thông tin, ĐHQG-HCM, TP. Hồ Chí Minh, Việt Nam

³Khoa Công nghệ Thông tin, Trường Đại học Nông Lâm, TP. Hồ Chí Minh, Việt Nam

I. VẤN ĐỀ VÀ ĐÓNG GÓP

Phát hiện xâm nhập ngay tại thiết bị biên: còn ba khoảng trống

Mô hình phát hiện xâm nhập nay đã *chạy được* trên bo mạch nhỏ, nhưng nếu một bo phải xét **mọi** luồng thì nó quá tải rất nhanh.

Nghiên cứu trước còn thiếu

1. Chỉ đo trên **một thiết bị**, không chia việc cho nhiều bo.
2. Hiếm khi báo **cùng lúc** tốc độ, độ trễ và năng lượng.
3. Điểm số offline lấy từ quy trình **rò rỉ dữ liệu**.

Câu hỏi

Có hai bo Jetson thì nên *đặt* việc phát hiện lên chúng thế nào?

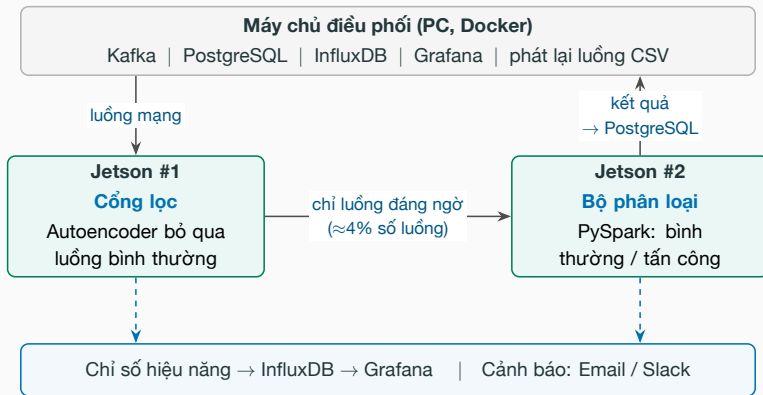
Cách làm

Tách làm hai: **cổng lọc** rẻ trên bo #1, **bộ phân loại** nặng trên bo #2, nối bằng Kafka.

1. Hệ thống phát hiện xâm nhập **hai tầng, phân tán**: cổng lọc và bộ phân loại Spark đặt trên hai bo khác nhau.
2. **Ba cách triển khai** trên cụm hai Jetson: tách tầng, nhân bản ngang, cụm Spark.
3. **Đo đầu cuối**: tốc độ, độ trễ, CPU/RAM từng bo và năng lượng mỗi phán quyết.
4. Mã nguồn, cấu hình và kịch bản đo **công khai**, chạy lại được.

II. HỆ THỐNG VÀ CÁCH ĐO

Kiến trúc: tách công lọc và bộ phân loại ra hai bo



Chỉ các luồng đáng ngờ mới đi qua Kafka sang bo #2.

Tầng 1 — cổng lọc (autoencoder)

- Học cách *tái tạo* lưu lượng bình thường.
- Luồng tái tạo tốt (sai số nhỏ) → coi là bình thường, **dừng luôn**.
- Luồng tái tạo kém → đáng ngờ, chuyển sang tầng 2.
- Ngưỡng sai số chọn *trước* trên 10% dữ liệu bình thường, không nhìn tập kiểm tra.

Tầng 2 — bộ phân loại (PySpark)

- Random Forest trên **29 đặc trưng** được SHAP chọn, không có cổng đích.
- Chỉ phải xét số ít luồng mà cổng chuyển sang.

Vì sao bộ phân loại là Random Forest trên Spark

- **Random Forest có sẵn trong Spark ML**, nên mô hình xuất ra chạy được trên Jetson (ARM64) mà không cần thư viện native như XGBoost.
- **Ít đặc trưng** (29 thay vì 77) thì dựng vector và suy luận nhanh hơn trên bo 8 GB.
- **Cùng một bộ công cụ** (Spark) vừa huấn luyện trên cụm vừa chạy trên thiết bị — cái giá của lựa chọn này ở Kết quả 5.

Ba cách triển khai, cùng một khối lượng việc

Một bo làm hết mọi việc — mốc so sánh.

A: Tách tầng cổng trên #1, phân loại trên #2.

B: Nhân bản mỗi bo làm đủ cả hai tầng, chia nhau luồng qua Kafka.

C: Cụm Spark PC làm master, hai bo làm worker.

Tốc độ tính bằng **số phán quyết mỗi giây**: một luồng được quyết ở cổng *hoặc* ở bộ phân loại, không đếm hai lần.

Thiết lập đo

2× Jetson Orin Nano Super: CPU 6 nhân, 8 GB RAM, GPU ≈ 67 TOPS

PySpark 3.4.1, Kafka 3.5, mạng Wi-Fi

Phát lại CICIDS2017, 100 luồng/giây

Mỗi cấu hình lặp ≥ 3 lần, bỏ giai đoạn khởi động

Năng lượng đo bằng tegrastats

III. KẾT QUẢ VÀ KẾT LUẬN

Kết quả 1: tách tầng nhanh gấp 24 lần, độ trễ gần như không đổi

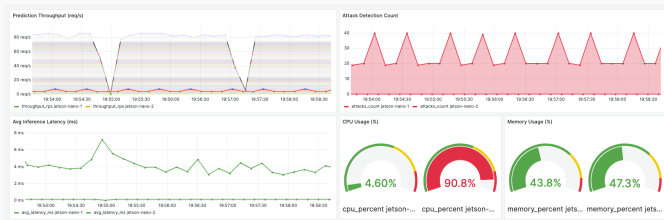
Cách triển khai	Phán quyết/giây	Độ trễ p95 (ms)
Một bo	2,6	12,3
A: Tách tầng	62,5	13,1
B: Nhân bản	5,9	21,6
C: Cụm Spark	90,0	23,9

p95: 95% số luồng có phán quyết nhanh hơn mức này.

- **Tách tầng: gấp 24 lần**, công xử lý xong 95,8% số luồng trước khi tới Spark.
- **Nhân bản: chỉ gấp 2,3 lần** — bo nào cũng vẫn phải chạy Spark.
- Cụm Spark nhanh nhất nhưng **dao động mạnh** ($\pm 25,6$).

Nên dùng tách tầng: nhanh và ổn định nhất.

Kết quả 2: nhìn trực tiếp thấy bo nào đang gánh việc



Lúc tải cao nhất, chế độ tách tầng:

- bo cổng lọc: **4,6% CPU**
- bo phân loại: **90,8% CPU**

Bộ phân loại chiếm gần hết sức tính — nên tách nó ra khỏi đường nhận dữ liệu là đáng.

Kết quả 3: năng lượng mỗi phán quyết giảm từ 2,5 J còn 0,18 J

Cách triển khai	J / phán quyết
Một bo	2,49
A: Tách tầng	0,18
B: Nhân bản	2,25
C: Cụm Spark	0,12

- Ở 100 luồng/giây, điện năng *tăng thêm* khi chạy quá nhỏ để đo ($< 0,15$ W).
- Nên con số chủ yếu là **điện nền của bo chia đều cho số phán quyết**.
- Tách tầng và cụm Spark ra nhiều phán quyết hơn 24–35 lần, nên mỗi phán quyết tốn ít hơn hẳn.

Đây là năng lượng ở *mức triển khai*, không phải năng lượng của riêng mô hình.

Kết quả 4: công lọc cho lọt bao nhiêu tấn công

Ngưỡng cổng (phân vị)	Tấn công chuyển tiếp	Báo nhầm	Luồng bỏ qua
0,99	74,2%	1,01%	88,0%
0,995 (đang dùng)	72,3%	0,53%	88,7%
0,999	58,1%	0,10%	91,2%

Đo offline trên tập kiểm tra CICIDS2017.

- Luồng bị cổng bỏ qua là *chốt luôn*: **khoảng 28% luồng tấn công** không tới được bộ phân loại.
- Hạ ngưỡng thì bắt thêm tấn công nhưng Spark phải xét nhiều luồng hơn.
- Người vận hành chọn điểm hợp với sức máy.

Tốc độ gấp 24 lần phải trả bằng recall: cổng là một **sự đánh đổi có thể chỉnh**, không phải bộ lọc miễn phí.

Kết quả 5: cái giá của việc dùng Spark để suy luận

Cùng một Random Forest, cùng 29 đặc trưng, cùng một bo — chỉ đổi bộ máy chạy:

Bộ máy chạy	Luồng/giây	p95 (ms)	RAM (%)	mJ mỗi luồng
PySpark (đang dùng)	22,6	532	24,3	71,6
scikit-learn	204,4	49,8	13,3	2,91
ONNX Runtime	7870,8	1,1	15,7	0,06

mJ mỗi luồng: năng lượng tăng thêm so với lúc bo nghỉ.

ONNX Runtime nhanh gấp **348 lần** PySpark. Hướng tiếp theo: [huấn luyện bằng Spark, suy luận bằng ONNX/TensorRT](#).

Những gì đã chỉ ra

- Tách cổng lọc và bộ phân loại ra hai bo: **2,6** → **62,5 phán quyết/giây**, p95 $\approx$ 13 ms.
- Năng lượng mỗi phán quyết: **2,5 J** → **0,18 J**.
- Đổi lại, cổng cho lọt $\approx$ **28%** luồng tấn công ở ngưỡng đang dùng.
- Spark là nút thắt suy luận: ONNX nhanh gấp 348 lần.

Hạn chế

- Hai bo, mạng Wi-Fi, dữ liệu phát lại.
- GPU chưa dùng; chưa thử tấn công né tránh.

Hướng tiếp theo

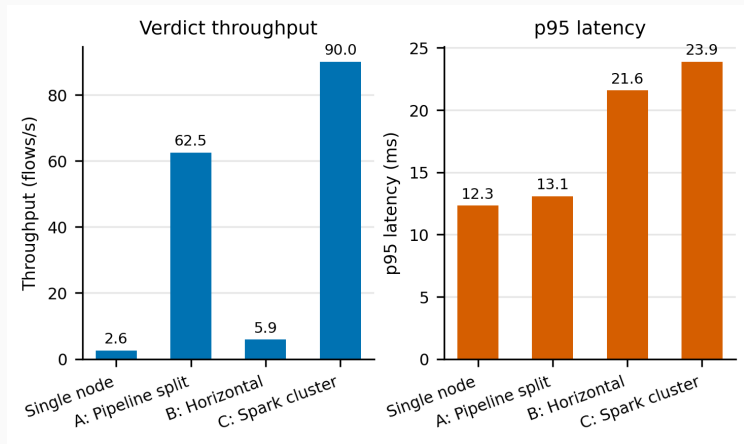
- Suy luận bằng ONNX/TensorRT.
- Lưu lượng thật; huấn luyện lại khi dữ liệu trôi.

Xin cảm ơn — kính mời câu hỏi

- [1] Sharafaldin, I., Lashkari, A. H. & Ghorbani, A. A. *Toward generating a new intrusion detection dataset and intrusion traffic characterization*. ICISSP 2018. **CICIDS2017**
- [2] Shi, W. et al. *Edge computing: Vision and challenges*. IEEE Internet of Things Journal 3(5), 2016. **điện toán biên**
- [3] Chandola, V., Banerjee, A. & Kumar, V. *Anomaly detection: A survey*. ACM Computing Surveys 41(3), 2009. **cổng lọc**
- [4] Lundberg, S. M. & Lee, S.-I. *A unified approach to interpreting model predictions*. NeurIPS 2017. **SHAP**
- [5] Zaharia, M. et al. *Spark: Cluster computing with working sets*. HotCloud 2010. **Spark**
- [6] Kreps, J., Narkhede, N. & Rao, J. *Kafka: a Distributed Messaging System for Log Processing*. NetDB 2011. **Kafka**
- [7] Mohan Kiran Kumar, K. et al. *Distributed Intrusion Detection System using Kafka and Spark Streaming*. ICVADV 2025. **hệ thống gần nhất**
- [8] Arp, D. et al. *Dos and Don'ts of Machine Learning in Computer Security*. USENIX Security 2022. **chống rò rỉ**

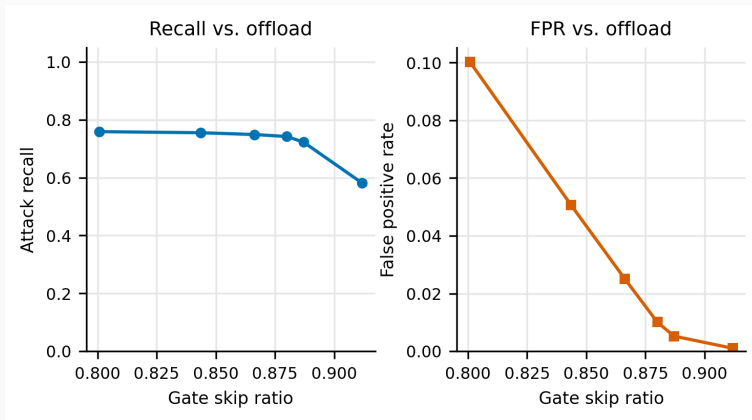
Slide dự phòng

Dự phòng: biểu đồ so sánh các cách triển khai



Số phán quyết mỗi giây (trái) và độ trễ p95 (phải).

Dự phòng: đường cong vận hành của cổng lọc



Recall tấn công (trái) và tỷ lệ báo nhầm (phải) theo tỷ lệ luồng bị bỏ qua. Bỏ qua nhiều hơn thì Spark nhẹ hơn nhưng lọt nhiều tấn công hơn.

Dự phòng: bảng điều khiển giám sát đầy đủ



Tốc độ tăng bo, số tấn công phát hiện, độ trễ suy luận, CPU/RAM hai bo, cảnh báo lưu trong PostgreSQL.

Dự phòng: quy trình đo

- Máy chủ khởi động các vai trò trên Jetson qua SSH, tạo tải, thu log từng bo.
- Mỗi cấu hình lặp ≥ 3 lần; bỏ giai đoạn khởi động để JVM chưa nóng máy không làm phồng số liệu.
- Truy vấn chỉ số khớp đúng cửa sổ tải, không lấy khoảng thời gian trượt.
- p95 tính theo từng bo trên dữ liệu thô của từng luồng; p95 của cụm lấy bo tệ nhất.
- Độ trễ đầu cuối tính từ lúc gửi tới lúc có phản quyết, đồng hồ đồng bộ NTP, không tính thời gian ghi CSDL.
- Năng lượng: tegrastats, 30 giây nền rồi cửa sổ tải; chế độ hai bo cộng cả hai bo.